C++ Dynamic Memory & Smart Pointers

Comprehensive Review Sheet: Stack vs Heap, Raw Allocation, RAII, Smart Pointers & Data Structures

1 Foundation: Stack vs. Heap Memory

Feature	Stack Memory	Heap Memory
Management	Automatic (managed by CPU / compiler scope).	Manual (via `new` / `delete` or Smart Pointers).
Lifetime	Limited to enclosing block/function scope.	Persists until explicitly freed or program ends.
Size & Speed	Fixed small size (usually 1–8 MB); extremely fast allocation.	Large flexible size (system RAM); slower allocation overhead.
Structure	Contiguous LIFO (Last-In, First-Out) stack frame structure.	Fragmented memory pool (dynamically requested from OS).

2 Legacy C++ Allocation (`new` & `delete`)

Raw dynamic memory allocation directly asks the OS for memory blocks. It requires manual cleanup and is prone to runtime errors.

Single Object Allocation

```cpp
// 1. Allocate on Heap
int* p = new int(42);

// 2. Access / De-reference
std::cout << *p;

// 3. Free memory
delete p;

// 4. Prevent dangling pointer
p = nullptr;
```

Dynamic Array Allocation

```cpp
// 1. Allocate Array on Heap
int* arr = new int[5]{10, 20, 30, 40, 50};

// 2. Access by index
std::cout << arr[2]; // 30

// 3. Free MUST use delete[]
delete[] arr;

// 4. Clear pointer
arr = nullptr;
```

Major Raw Pointer Pitfalls:

- **Memory Leak:** Forgetting to call `delete` leaves allocated memory inaccessible until the process terminates.
- **Dangling Pointer:** Accessing memory via a pointer after calling `delete` on it.
- **Double Free:** Calling `delete` on the same address twice leads to program crash or corruption.
- **Mismatched Delete:** Calling `delete` instead of `delete[]` on an array causes undefined behavior.

3 Modern C++ Smart Pointers (RAII)

Smart pointers in `<memory>` wrap raw pointers using **RAII (Resource Acquisition Is Initialization)**. They clean up memory automatically when the smart pointer object goes out of scope.

A. `std::unique_ptr<T>` — Exclusive Ownership

Only **one** `unique_ptr` can own a resource at any time. Copying is disabled (compile error), but ownership can be transferred via `std::move()`.

```
#include <memory>

void uniqueExample() {
    // Preferred creation: std::make_unique (C++14)
    auto ptr1 = std::make_unique<int>(100);

    // auto ptr2 = ptr1; // ❌ COMPILE ERROR! Copying forbidden.

    // Transfer ownership to ptr2
    std::unique_ptr<int> ptr2 = std::move(ptr1);

    // ptr1 is now nullptr; ptr2 owns the integer 100
    if (!ptr1) {
        std::cout << "ptr1 is now null
";
    }
} // ptr2 goes out of scope HERE -> Memory automatically freed!
```

B. `std::shared_ptr<T>` — Shared Ownership

Multiple `shared_ptr` instances can share access to the same heap object. It uses reference counting internally; memory is freed only when the last reference is destroyed.

```
void sharedExample() {
    // Allocation with control block for ref count
    auto p1 = std::make_shared<int>(500);

    {
        auto p2 = p1; // ✅ Copying allowed! Ref count = 2
        std::cout << "Ref Count: " << p1.use_count() << "
"; // Outputs 2
    } // p2 destroyed -> Ref count drops to 1

    std::cout << "Ref Count: " << p1.use_count() << "
"; // Outputs 1
} // p1 destroyed -> Ref count becomes 0 -> Object DELETED!
```

C. `std::weak_ptr<T>` — Non-Owning Observer

Holds a weak (non-owning) reference to an object managed by `shared_ptr`. Does **not** increase reference count. Used to break circular dependencies and for caching.

```cpp
void weakExample() {
    std::shared_ptr<int> shared = std::make_shared<int>(99);
    std::weak_ptr<int> weak = shared; // Ref count remains 1

    // To access data, convert weak_ptr to temporary shared_ptr via lock()
    if (auto temp = weak.lock()) {
        std::cout << "Value: " << *temp << "
"; // 99
    }

    shared.reset(); // Destroys original object

    if (auto temp = weak.lock()) {
        // Won't execute
    } else {
        std::cout << "Resource has been destroyed!
";
    }
}
```

4 Data Structures: Raw vs. Smart Pointer Conversions

1. Singly Linked List Node

```cpp
// --- RAW POINTER WAY ---
struct RawNode {
    int val;
    RawNode* next;
    RawNode(int v) : val(v), next(nullptr) {}
};
// Deletion requires manually traversing and calling delete on every node!

// --- MODERN SMART POINTER WAY ---
struct SmartNode {
    int val;
    std::unique_ptr<SmartNode> next; // Unique ownership of next node
    SmartNode(int v) : val(v), next(nullptr) {}
};
// Automatic recursive cleanup when head unique_ptr goes out of scope!
```

2. Binary Tree Node

```cpp
struct TreeNode {
    int value;
    std::unique_ptr<TreeNode> left;
    std::unique_ptr<TreeNode> right;

    TreeNode(int v) : value(v), left(nullptr), right(nullptr) {}
};

int main() {
    auto root = std::make_unique<TreeNode>(10);
    root->left = std::make_unique<TreeNode>(5);
    root->right = std::make_unique<TreeNode>(15);
    // Top-level root destruction automatically deletes entire left & right subtrees
}
```

3. Graph Node (Handling Cycles with `weak_ptr`)

> **Why `weak_ptr` is required in Graphs:** If Node A has a `shared_ptr` to Node B, and Node B has a `shared_ptr` to Node A, both reference counts remain `1` forever — causing a severe **Circular Memory Leak**. Using `weak_ptr` for back-edges breaks the cycle.

```
struct GraphNode {
    int id;
    std::vector<std::shared_ptr<GraphNode>> neighbors;   // Forward edges
    std::vector<std::weak_ptr<GraphNode>> back_edges;   // Prevents cycles!

    GraphNode(int id) : id(id) {}
};

void createGraph() {
    auto nodeA = std::make_shared<GraphNode>(1);
    auto nodeB = std::make_shared<GraphNode>(2);

    nodeA->neighbors.push_back(nodeB);  // A points to B (shared ownership)
    nodeB->back_edges.push_back(nodeA); // B points to A via weak_ptr (no cycle leak!)
}
```

5 Cheat Sheet & Decision Matrix

Construct	Header	Ownership	Copy / Move	When to Use
`std::unique_ptr`	`<memory>`	Single / Exclusive	Move Only	Default choice for trees, lists, and exclusive dynamic objects.
`std::shared_ptr`	`<memory>`	Shared (Ref Counted)	Copyable & Moveable	Resources shared across multiple objects or graph nodes.
`std::weak_ptr`	`<memory>`	Non-Owning Observer	Copyable & Moveable	Cache systems, parent pointers in double linked lists/graphs.
`std::make_unique()`	`<memory>`	Factory Helper	N/A	Creates `unique_ptr` safely without explicit `new`.
`std::make_shared()`	`<memory>`	Factory Helper	N/A	Single allocation for object + control block (faster).
`std::vector<T>`	`<vector>`	Contiguous Array	Copyable & Moveable	Prefer over `new T[]` or `unique_ptr<T[]>` for dynamic arrays.

> **Golden Rules of Modern C++ Memory Management:**
>
> 1. **Rule of Zero:** Prefer standard containers (`std::vector`, `std::string`) and smart pointers so you never need custom destructors.
> 2. **Avoid Raw `new` / `delete`:** Always construct with `std::make_unique` or `std::make_shared`.
> 3. **Pass Smart Pointers Correctly:** Pass by value to share/transfer ownership; pass raw reference (`T&`) or raw pointer (`T*` via `.get()`) for temporary observation without ownership change.